

Here are the technical notes and tasks formulated in English, ready to be handed over to the development team for implementation:

Student UI & Time Controls Updates

1. Pre-Exam Instructions View

- **Description:** Implement an intermediate screen that appears after the general countdown timer ends and before the student's actual exam time begins.
- **Backend Update:** Add a new `instructions` field (Type: `Rich Text/HTML`) to the `Exam` table to allow instructors to insert and format specific guidelines (e.g., "Calculators are prohibited," "Ensure your camera is active").
- **Frontend Update:**
 - Route the student to the "Instructions" view immediately after the pre-exam countdown concludes.
 - Render a prominent **"Start Exam"** action button below the instructions.
 - **Critical Logic:** The actual exam countdown (`Duration Timer`) must *strictly* start only upon clicking this button, not a second before.

2. Availability Window vs. Time Limit (Duration)

- **Description:** Programmatically differentiate between the time frame an exam is visible/accessible and the actual time limit granted to the student to complete it.
- **Database Verification:** Ensure the distinct separation of the following fields:
 1. `availableFrom` (Timestamp): When the exam opens for entry.
 2. `availableTo` (Timestamp): When the system definitively closes the exam.
 3. `durationMinutes` (Integer): The allowed completion time limit (e.g., 60 minutes).
- **Auto-Submit Logic:** The system must force an auto-submit of the attempt when **either of the following conditions is met (whichever occurs first)**:
 1. The student's `durationMinutes` expires.
 2. The global `availableTo` timestamp is reached. (*Example: If the exam has a 60-minute duration, closes at 12:00 PM, and the student starts at 11:30 AM, the system must auto-submit their attempt in exactly 30 minutes*).

3. Item-Level Time Limit (Optional)

- **Description:** Introduce an advanced capability to restrict time per individual question, rather than just the overall assessment.
- **Backend Update:** Add a `timeLimitSeconds` field (Type: `Integer, Nullable`) to the `Item` table or the exam-item configuration payload.
- **Frontend Update:**
 - If a value exists, render a dedicated mini-countdown timer adjacent to the specific question.

- Upon reaching 00:00, the UI must automatically save the current response (if any) and force an auto-advance to the next item.
- The "Previous" navigation button must be locked or disabled for time-expired items to prevent retroactive modifications.

4. Optional AI Proctoring (Toggle Feature)

- **Description:** Provide the flexibility to create and conduct low-stakes exams without enforcing AI proctoring, camera tracking, or biometric verification.
- **Backend Update:** Introduce a boolean flag such as `isProctoringEnabled` (Type: `Boolean`, Default: `true`) within the `Exam` table, or append it to the existing `settings` (JSONB) column.
- **Frontend Update:**
 - Add a clear toggle switch in the "Create Exam" setup wizard labeled "Enable AI Proctoring".
 - If toggled off (`false`), the student's entry flow must completely bypass the camera/microphone permission checks and facial recognition steps.
 - The frontend application must not initialize the AI proctoring engine or background monitoring processes during the active attempt.

5. Multi-Tiered Item Bank & Role-Based Access Control (RBAC)

Description: Evolve the Item Bank module to support a dual-level ownership and collaboration architecture. The system must support top-down institutional repositories and peer-to-peer collaborative instructor banks.

1. Database Schema Enhancements:

- **ItemBank Table Adjustments:**
 - Add `bankLevel` (Enum: `INSTITUTIONAL`, `PERSONAL`).
 - Add `ownerId` (Foreign Key). If `INSTITUTIONAL`, this points to the Institution ID. If `PERSONAL`, it points to the specific Instructor's User ID.
- **Create ItemBankAccess (Junction Table):**
 - This table manages the many-to-many sharing permissions.
 - **Fields:** `bankId` (FK), `userId` (FK), `permissionRole` (Enum: `OWNER`, `EDITOR`, `VIEWER`), and `assignedBy` (FK).

2. Backend Business Logic & Workflows:

- **Level 1: Institutional Banks (Top-Down):**
 - System Administrators or Department Heads can create an `INSTITUTIONAL` bank.

- They can assign multiple specific instructors to this bank as EDITORs (via the ItemBankAccess table).
- Assigned instructors can add, edit, or categorize questions, but they cannot delete the bank or alter its core settings.
- **Level 2: Personal Banks (Peer-to-Peer):**
 - Any instructor can create a PERSONAL bank (becoming the OWNER).
 - Create an endpoint (POST /api/item-banks/{id}/collaborators) allowing the owner to grant access to other instructors within the same institution.
 - The owner can assign either EDITOR rights (can add/modify items) or VIEWER rights (can only pull items for their exams).

3. Frontend UI / UX Updates:

- **Categorized Repository Dashboard:** Organize the Item Bank screen into three distinct tabs for clear navigation:
 - *Institution Banks* (Managed by the department).
 - *My Private Banks* (Owned by the active user).
 - *Shared with Me* (Collaborative banks owned by peers).
- **Collaboration Management Component:** Build a modal/dialog for Personal Banks labeled "Manage Access" or "Invite Colleagues". This UI should allow the instructor to search for peers by name or email and assign their respective permission roles.

Create New Exam > Exams

Settings 4 — Questions 3 — AI Generation 2 — Basic Info ✓

Generate Questions with AI

[Upload file \(.pdf, .doc, .docx, .txt, .md\)](#) 

Paste or upload document content

Paste your lecture notes, textbook excerpt, or topic description here — or upload a PDF / Word / .txt file
...above

Difficulty

Medium ▼

Question Type

Multiple Choice (MCQ) ▼

Generate 5 Questions 

> Next

Back <

6. UI/UX Refactoring: Decoupling AI Generation from Exam Wizard

Description: Migrate the AI Question Generation module completely out of the "Create New Exam" wizard and embed it into the "Item Bank" management module. The Exam creation process will now exclusively rely on selecting pre-existing items from populated banks.

1. Exam Wizard Streamlining (Frontend):

- **Remove Step 2:** Completely eliminate the "AI Generation" step (shown in [image](#)) from the exam creation stepper.
- **Update Stepper Flow:** The new logical progression must be simplified to: [Step 1: Basic Info](#) -> [Step 2: Select Questions \(Cross-Bank Selector\)](#) -> [Step 3: Settings](#).

2. Item Bank AI Integration (Frontend):

- **Relocate UI Component:** Move the entire AI generation interface (Upload file, Document content text area, Difficulty dropdown, Question Type dropdown, and the "Generate Questions" button) into the individual Item Bank view.

- **Entry Point:** Add a primary action button labeled "Generate with AI" or "Import via AI" inside the Item Bank detail page alongside the manual "Add Question" button.

3. Backend API & Data Flow Adjustments:

- **Endpoint Modification:** Update the AI generation API endpoint to accept `itemBankId` in its payload instead of an `examId`.
- **Direct Saving:** Once the AI model returns the generated questions, they must be committed directly to the `Item` table with the corresponding `itemBankId` as their foreign key.
- **State Update:** The frontend must immediately refresh the bank's item list to display the newly generated questions for the instructor to review, edit, or approve before they are ever used in an exam.

Generate Questions with AI

Upload file (.pdf, .doc, .docx, .txt, .md) 

Paste or upload document content

Paste your lecture notes, textbook excerpt, or topic description here — or upload a PDF / Word / .txt file
...above

Difficulty

Medium 

Question Type

Multiple Choice (MCQ) 

Generate 5 Questions 

Advanced AI Item Generation (CLO-Aware & Batch-Controlled)

Overview for Development Team: Refactor the AI Generation component to support dynamic quantity selection with strict server-side limits, and integrate explicit Course Learning Objective (CLO) mapping directly into the AI prompt pipeline to ensure institutional compliance.

1. Frontend Component Updates (AI_Generator_UI)

- **CLO Integration:** Introduce a **Select** dropdown labeled "Target CLO". This must fetch and display the associated learning objectives from the **LearningObjective** table.

The selected **CLO_ID** must be included in the submission payload.

- **Dynamic Quantity Input:** * Introduce an **InputNumber** field with **min={1}** and **max={MAX_BATCH_SIZE}** (e.g., 15).
 - The primary action button must be reactive, displaying: **Generate {n} Questions** based on the user's input.
 - Implement client-side validation to block submissions exceeding the maximum batch size.

2. Backend API & LLM Pipeline Enhancements

- **Payload Validation:** The API endpoint handling the generation must enforce a strict server-side validation rejecting any request where **quantity > MAX_BATCH_SIZE**.
- **Contextual Prompt Engineering:** * The backend must resolve the provided **CLO_ID** to its actual textual description prior to calling the LLM.
 - **Prompt Modification:** Append a strict directive to the system prompt: *"The generated questions must strictly align with and accurately assess the following Course Learning Objective: [Inject CLO Text]. Ensure distractors reflect common student misconceptions related to this specific objective."*
- **Database Mapping:** Upon successful generation, the backend must automatically save the generated items into the **Item** table, ensuring the **learningObjectiveId** foreign key is correctly populated for every item in the batch.



Dynamic Question Pooling

Draw a random subset of questions from a larger pool at runtime (Phase 2 feature).

Pool size (total items)

30

Questions per student

20

Phase 2: at exam start, the system will randomly pick 20 questions from a pool of 30 approved items matching this exam's subject/difficulty.

Stratified Dynamic Pooling & Test Blueprint Integration

Overview for Development Team:

The simple random pooling feature must be upgraded to a "Stratified Random Sampling" engine based on Course Learning Objectives (CLOs). This ensures every dynamically generated exam adheres strictly to the academic Test Blueprint, maintaining construct validity for accreditation compliance.

1. Frontend Component (**DynamicPooling_Blueprint_UI**)

- **Replace Basic Inputs:** Remove the static **Questions per student** input shown in the current UI.
- **Render Blueprint Matrix:** Build a dynamic table that lists all unique CLOs available within the selected Item Bank(s).
 - **Column 1 (CLO Name/Code):** Fetch from **LearningObjective** table.
 - **Column 2 (Available Pool):** Display the count of approved items mapped to this specific CLO.
 - **Column 3 (Target Draw):** An **InputNumber** field where the instructor defines exactly how many questions the system should randomly draw *from this specific CLO*.
- **Validation:** **Target Draw** must be $\leq$ **Available Pool** for each row. The total exam length is automatically calculated as the sum of all **Target Draw** inputs.

2. Backend Logic & Stratified Sampling Algorithm

- **Payload Structure:** The **Exam** settings JSONB field must now store a blueprint mapping object instead of a single integer. Example:
`"dynamicPoolingBlueprint": { "CLO_01": 5, "CLO_02": 10, "CLO_03": 5 }`
- **Runtime Exam Generation (JIT Assembler):**
 When a student starts the exam, the backend engine must execute a stratified query:
 1. Group the available item pool by **learningObjectiveId**.
 2. For each group, execute a randomized query (e.g., **ORDER BY RANDOM()**) and slice the array based on the integer defined in the **dynamicPoolingBlueprint**.
 3. Concatenate the results into a single exam payload, apply one final shuffle to mix the CLOs, and serve it to the student.

Multi-Section Exam Architecture

Overview: Transition the core exam engine from a single-layer structure to a hierarchical, multi-section architecture. This allows for isolated timers, section-specific instructions, independent sequential locking, and granular scoring models, aligning the platform with high-stakes international testing standards.

1. Database Schema Enhancements (Backend)

- **New Table: ExamSection**
 - **id** (UUID, Primary Key).
 - **examId** (Foreign Key -> **Exam** table).
 - **title** (String): e.g., "Reading Comprehension".
 - **instructions** (Rich Text/HTML).
 - **durationMinutes** (Integer, Optional): Specific time limit for this section.
 - **orderIndex** (Integer): Controls the sequence of sections.

- **sectionWeight** (Decimal/Float): e.g., 0.40 for 40% of the total exam score.
- **passingThreshold** (Integer, Optional): Minimum score required to pass this specific section.
- **Table Update: Item / ExamItem**
 - Modify associations so items (questions) map directly to a **sectionId** instead of the root **examId**.
- **Table Update: Exam (Settings Payload)**
 - Add **isSectionSequential** (Boolean, Default: **false**): Locks previous sections upon completion.
 - Add **isItemSequential** (Boolean, Default: **false**): Locks previous questions within the active section.

2. Exam Builder (Instructor UI)

- **Section Management:**
 - The builder must allow instructors to "Add New Section", assigning a title, specific instructions, and a dedicated timer (**durationMinutes**) for each.
 - Items selected from the Item Bank must be assigned/dropped into specific sections.
- **Sequential Locking Controls:**
 - Provide two distinct toggles in the settings panel:
 1. **Lock Completed Sections:** Prevents backward navigation across sections.
 2. **Lock Answered Questions:** Prevents backward navigation between individual items within a section.
- **Scoring Configuration:**
 - UI inputs to assign a specific weight (%) to each section, ensuring the total sum equals 100%.
 - Optional input to define a minimum passing threshold per section.

3. Exam Execution & Delivery (Student UI)

- **Interstitial Instruction Screens:**
 - Students do not start the exam directly. They land on the "Section 1 Instructions" screen.
 - The specific timer for Section 1 strictly begins only when the student clicks "Start Section".
- **Isolated Timers & Auto-Advance:**
 - The visible countdown reflects the active section's **durationMinutes**.
 - Upon reaching **00:00**, the system auto-submits the section and routes the student to the "Section 2 Instructions" screen (waiting area).
- **Independent Navigation Logic:**
 - If **isSectionSequential=true**: Submitting a section permanently disables

access to it.

- If `isItemSequential=true`: Submitting an answer auto-advances to the next question, hiding/disabling the "Previous" button entirely, regardless of section rules.

4. Scoring Engine & Reporting (Backend & Analytics)

- **Hierarchical Calculation Logic:**
 - **Raw Score:** Calculate the sum of correct items per section.
 - **Scaled Score:** Normalize the raw score for independent reporting (e.g., Section 1: 85/100).
 - **Composite Total:** Calculate the final grade by applying the configured weights:
$$\text{TotalScore} = \sum (\text{SectionScaledScore} \times \text{SectionWeight})$$
- **Validation Check:** If a `passingThreshold` is set and not met in *any* section, the overall exam status is flagged as "Failed" regardless of the TotalScore.
- **Diagnostic Reporting:**
 - Student dashboards and PDF certificates must display the breakdown: Overall Score, followed by individual Section Scores and Pass/Fail statuses.

Update 12 july

Advanced Item Types & Multimedia Responses

Overview for Development Team: Expand the **EvaLiX** item bank and exam engine to support advanced, multimedia-driven, and composite assessment formats. This requires implementing new UI response components (Audio/Video), integrating cloud storage logic for heavy media files, and developing a dynamic split-screen layout for complex case-study (composite) questions.

1. Database Schema Enhancements (Item Table)

- **itemType (Enum Expansion):** Add new types: **AUDIO_RESPONSE**, **VIDEO_RESPONSE**, and **COMPOSITE_CASE**.
- **New JSONB Column: mediaSettings:**
 - **minDurationSeconds** (Integer): Minimum required recording time to prevent blank submissions.
 - **maxDurationSeconds** (Integer): Maximum allowed recording time.
 - **prepTimeSeconds** (Integer): Countdown time allowed for the student to read/prepare before auto-recording starts.
 - **allowScreenShare** (Boolean): Specific to Video Responses; enables dual-capture (Screen + Webcam).
 - **maxRetries** (Integer): Number of times a student can delete and re-record their answer.
- **Composite Question Mapping (ExamItem Table):**
 - Add **parentItemId** (UUID, Nullable): If an item is a sub-question belonging to a case study, this field links it to the main **COMPOSITE_CASE** item.

2. Item Type 1: Audio Response (**AUDIO_RESPONSE**)

- **UI Components:**
 - **Preparation Timer:** A countdown clock (e.g., 60 seconds) that starts immediately when the question loads.
 - **"Skip Prep" Button:** Allows the student to bypass the wait and start recording immediately.
 - **Live Waveform Visualizer:** Implement Web Audio API to show a real-time waveform during recording, assuring the student that the microphone is capturing audio.
 - **Auto-Start Logic:** If the prep timer hits **00:00**, the UI must automatically trigger the microphone and begin the recording up to the **maxDurationSeconds**.
- **Storage & Handling:** Compress audio on the client-side (e.g., MP3/WebM) before uploading to the designated cloud storage bucket (e.g., Google Cloud Storage/Firebase).

Storage).

3. Item Type 2: Video Response (**VIDEO_RESPONSE**)

- **UI Components:**
 - Shares the exact Preparation Timer and Auto-Start logic as the Audio type.
 - **Camera Preview:** A live video feed box for the student to check their framing.
 - **Screen Share Toggle (Optional):** If `allowScreenShare=true`, prompt the browser's screen-sharing API before recording starts, recording a Picture-in-Picture (PiP) or side-by-side video.
- **Storage & Handling:**
 - **Resolution Limiter:** Force the browser's `getUserMedia` constraints to a maximum of 480p or 720p (e.g., `video: { width: 1280, height: 720 }`) to minimize payload size and server costs.

4. Item Type 3: Composite / Case-Study (**COMPOSITE_CASE**)

- **Layout & UI Engine (Split-Screen Architecture):**
 - **Directionality Logic (RTL/LTR):**
 - If the Exam/Item language is **Arabic (RTL)**: The parent passage/media (Stem) is fixed on the **Right** pane, and the scrollable child questions are on the **Left** pane.
 - If the Exam/Item language is **English (LTR)**: The parent passage/media is fixed on the **Left** pane, and the scrollable child questions are on the **Right** pane.
 - **Independent Scrolling:** The parent pane must remain sticky/fixed while the user scrolls through the child items in the adjacent pane.
- **Child Item Flexibility:**
 - The child pane must be capable of rendering *any* existing item type (MCQ, Essay, Audio, Video) sequentially.
- **Scoring Logic:**
 - The total score of the **COMPOSITE_CASE** is strictly calculated as the automated sum of its child questions.



ExamPro

MENU



Dashboard



Exams



Item Bank



Live Monitor



Students



Analytics



Settings

Add CLO-Mapping to teacher portal.

Scoring Rubric

Dimension	Excellent (4)	Good (3)	Fair (2)	Poor (1)	Weight %
Content Accuracy	Describe...	Describe...	Describe...	Describe...	25
Critical Thinking	Describe...	Describe...	Describe...	Describe...	25
Structure & Clarity	Describe...	Describe...	Describe...	Describe...	25
Evidence & Examples	Describe...	Describe...	Describe...	Describe...	25

AI-Powered Hierarchical Rubric Engine

Overview: This document outlines the architectural and functional requirements for developing a dynamic, AI-assisted scoring rubric module. The engine transitions the platform from flat grading grids to an intelligent, hierarchical assessment tool capable of automating complex evaluations while maintaining strict academic oversight.

1. Hierarchical Rubric Architecture & Structural Requirements

- **Nested Architecture (Tree Structure):** The system must support a multi-level hierarchy where primary dimensions (Parent Nodes) can branch into specific sub-dimensions (Leaf Nodes) without rigid depth constraints.
- **Dynamic Customization & Labeling:** Instructors must have complete freedom to rename and customize all elements, including: Parent/Sub-dimension titles, performance level headers (e.g., Excellent, Proficient, Developing), and the detailed qualitative performance descriptors within each intersecting cell.
- **Real-Time Auto-Validation Engine:** * The system must enforce strict mathematical validation: the sum of the weights of all sub-dimensions must exactly equal the weight of their parent dimension ($\sum \text{Weightsub} = \text{Weightparent}$).
 - The sum of all root dimensions must exactly equal 100%.
 - The UI must dynamically lock the "Save Rubric" action and highlight discrepancies in red if the weight distribution is imbalanced.
- **Granular CLO Mapping:** The system must allow mapping individual leaf nodes (or childless parent dimensions) to specific Course Learning Objectives (CLO_ID). This ensures that a single complex question can assess and report on multiple distinct skills

simultaneously.

2. AI & LLM Grading Engine Integration

- Context Assembly Pipeline: Upon submission of a student's attempt (whether text or transcribed audio/video), the backend must assemble a comprehensive payload for the LLM. This payload must include:
 - The core question stem and any associated multimedia/reference text.
 - The student's complete response.
 - The full hierarchical rubric JSON, containing all dimensions, levels, and explicit performance descriptors.
- Prompt Engineering & Governance Constraints:
 - The System Prompt must instruct the LLM to act as a strict, unbiased academic evaluator.
 - The LLM must be tightly constrained to evaluate the response *strictly* against the provided cell descriptors, preventing hallucinations or arbitrary scoring outside the defined rubric parameters.
- Structured Output Requirement (JSON): The LLM must return a strictly formatted, machine-readable JSON object containing:
 - The evaluated **Sub-Dimension ID**.
 - The awarded performance level.
 - Academic Rationale: A concise, evidence-based justification explaining why this specific level was chosen, referencing exact phrases from both the student's answer and the rubric descriptor.
- Zero-Anchor / Veto Handling: If a specific dimension is flagged by the instructor as a "Veto" criterion (e.g., Academic Integrity / Plagiarism), and the AI evaluates this dimension at a zero-score level, the system must automatically override all other dimensions and nullify the total score for that item.

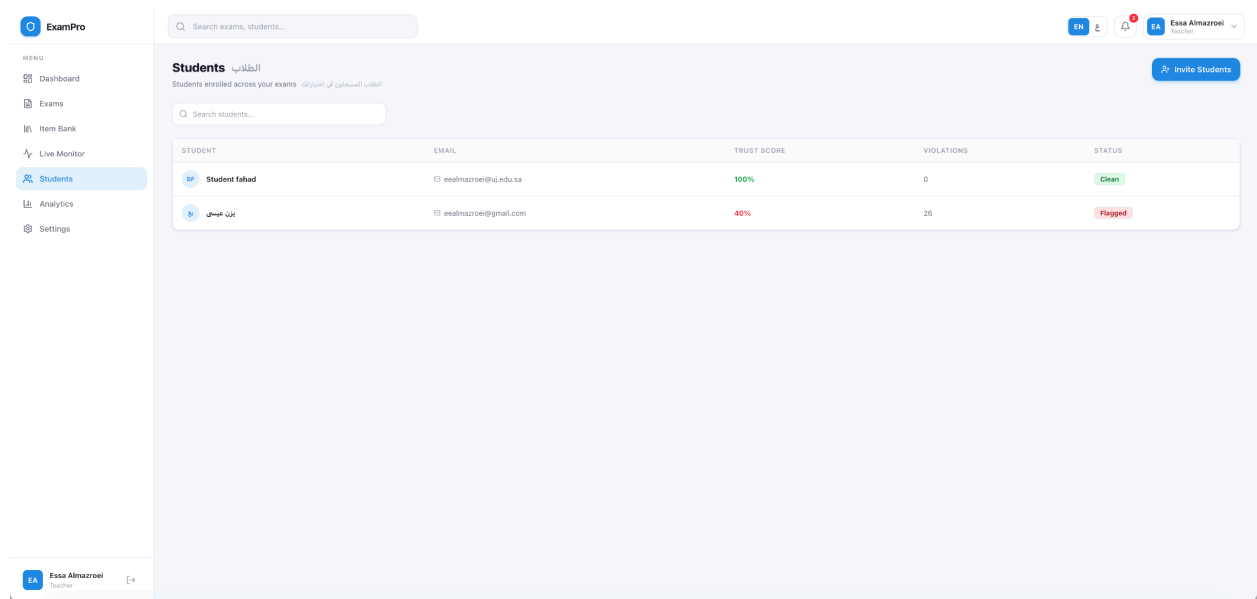
3. UI/UX Workflow & Human-in-the-Loop (HITL)

- Draft/Pending State: AI-generated grades and rationales must never be published to students automatically. They must be securely stored in a "Pending Review" state.
- Instructor Audit Dashboard: Build a dedicated split-screen grading interface. The student's response is displayed alongside the interactive rubric. The AI's selected cells must be clearly highlighted, with the AI's generated rationale displayed directly beneath the relevant dimension.
- Manual Override & Recalculation: The instructor maintains ultimate authority. Clicking any alternative cell in the rubric instantly overrides the AI's selection. The system must immediately recalculate the overall score based on this human input and flag the dimension as "Manually Modified."
- Bulk Approval Workflow: To optimize instructor workflow, provide a one-click "Approve All" action to officially publish all unmodified AI-graded responses, transitioning them

from "Draft" to "Finalized."

4. Data Governance & Academic Analytics

- System Audit Logging: The platform must log operational metrics for each AI transaction, including processing latency, API token consumption, and the modification rate (how often instructors override the AI). This data is critical for fine-tuning prompts and measuring LLM alignment.
- KPIs & Quality Profiling (Accreditation Readiness): Upon final grade approval, the reporting engine must aggregate the scores based on the mapped **CLO_IDs**. This enables the automated generation of granular, quantitative academic quality reports. This precise breakdown of student strengths and weaknesses at the sub-skill level ensures seamless compliance with institutional and NCAAA accreditation requirements.



Feature 1: Institutional Group & Cohort Management (Phase 1)

Overview: Implement a robust, group-based enrollment system to allow instructors to organize students into structural containers (e.g., Sections, Classes, Cohorts). Exams can then be assigned to an entire group with a single action.

1. Database Schema Enhancements

- New Table: **StudentGroup**
 - **id** (UUID, Primary Key).
 - **name** (String): e.g., "Section 101", "Cohort B".
 - **description** (Text, Optional).
 - **institutionId** (Foreign Key -> **Institution**).
 - **createdBy** (Foreign Key -> **User** / Instructor).
 - **createdAt** / **updatedAt** (Timestamps).
- New Junction Table: **GroupMembership**
 - This handles the many-to-many relationship between students and groups.
 - **id** (UUID, Primary Key).
 - **groupId** (Foreign Key -> **StudentGroup**).
 - **studentId** (Foreign Key -> **User** / Student).
 - **enrolledAt** (Timestamp).
- Update Table: **ExamAssignment**
 - Add **groupId** (Foreign Key -> **StudentGroup**, Nullable) to route exams directly to groups.

2. UI/UX Workflow (Instructor Dashboard)

- Group Creation Modal: A clean interface allowing instructors to create a group, name it, and add a brief description.
- Direct Student Addition to Groups: * Inside the group detail view, provide a button labeled "Add Students to Group".
 - Instructors can search for registered students by name or email and add them to the group list.
- Bulk CSV/Excel Group Import: * Provide a secure template upload mechanism. The system parses columns: **Student Name**, **Student Email**, and **Group Name**.
 - Backend Logic: The parser dynamically maps students to the corresponding groups, creating the group automatically if it does not already exist in the system.

3. Exam Targeting Logic

- When creating or publishing an exam, the instructor can select one or multiple groups from a dropdown menu. The backend resolution service instantly populates the exam access list for all active members belonging to those selected groups.

Feature 2: Dynamic Student Profile Tagging & Advanced Filtering (Phase 2)

Overview: Introduce a flexible metadata layer directly onto the student's profile. This allows instructors to tag individual students with specific attributes (e.g., academic status, accommodations, tracks) and utilize these tags for granular search, filtering, and cross-cohort exam targeting.

1. Database Schema Enhancements

- Update Table: **User / Student** (Profile Metadata):
 - Add a **tags** field (Array of Strings, e.g., ["**Postgraduate**", "**Scholarship**", "**Extra_Time**"]).
- Update Table: **ExamAssignment** (Targeting Criteria):
 - Expand the assignment model to support a **targetCriteria** JSONB object that includes both **groupIds** (from Phase 1) and **targetTags** (Phase 2).
 - Add an **operator** flag (Enum: **AND**, **OR**) to manage complex rule conditions.

2. UI/UX Workflow (Student Management Dashboard)

- Inline Profile Tagging: * Modify the main student list interface to include a dedicated "Tags" column.
 - Instructors can click on a student's row to view, add, or delete tags inline via a tag-chip component with autocomplete support.

- Bulk Tag Management: * Enable multi-select checkboxes on the student list. Instructors can select 20 students at once and click a top action button labeled "Bulk Tag" or "Remove Tag".
- Advanced Filtering Controls: * Add a filtering panel above the student list table, allowing instructors to filter the workspace view by group, specific tags, or combinations of both.

3. Advanced Exam Targeting Engine (Backend Logic)

- The exam publishing process is upgraded to evaluate the **targetCriteria** payload:
 - Conditional Intersection (**AND**): The system serves the exam *only* to students who belong to the selected Group AND possess the specific profile tag (e.g., Target: "Section 101" students who are also tagged as "Special Accommodations").
 - Conditional Union (**OR**): The system serves the exam to anyone in the selected Group OR anyone globally possessing that profile tag.

4. Analytical & Quality Reporting Capabilities

- The reporting engine can now aggregate assessment data using profile tags as the primary grouping variable. Administrators can extract macro performance profiles (e.g., comparing the average scaled score of "Scholarship" students versus "Non-Scholarship" cohorts across various exams), independent of their class groups.

Update 16 July 2026 (think with me)

UI/UX Wireframe Specifications:

EvaliX Exam Builder Wizard (Default)

Architecture Model: Multi-Page Application (MPA) Global UI Elements (Persistent across all 5 pages):

- Top Navigation Bar: Contains the "EvaliX" logo, User Profile, and a "Save & Exit" button.
- Progress Stepper: A horizontal visual indicator showing the 5 steps. The active step is highlighted. Previous steps are clickable (for backward navigation), while future steps are disabled until the current page is successfully saved and validated.
- Toast Notifications: To display "Draft Saved Successfully" when transitioning between pages.

Page 1: /step-1-configuration (Exam Configuration)

Purpose: Define the core identity and strict navigation rules of the exam.

- Layout Structure: Single column, centered form.
- UI Components:
 - Page Header: Title **Step 1: Basic Settings**.
 - Input Fields (Standard):
 - **Exam Title** (Text Input, Required).
 - **Course Code** (Text Input, Optional).
 - **Brief Description / General Instructions** (Rich Text Editor).
 - Global Timer Settings:
 - **Total Exam Duration** (Number Input - Minutes). *Note: This is disabled if the instructor chooses to set individual timers per section in Step 2.*
 - Sequential Locking Panel (Card Container):
 - Title: **Security & Navigation Rules**.
 - **Lock Completed Sections** (Toggle Switch): Tooltip explains that students cannot return to a previous section once submitted.
 - **Lock Answered Questions** (Toggle Switch): Tooltip explains that

students cannot navigate backward between items within the same section.

- Bottom Action Bar: Sticky at the bottom.
 - Primary Button: **Save & Continue ->**

Page 2: /step-2-sections (Section Architecture)

Purpose: Build the hierarchical structure of the exam to support granular scoring and independent section rules.

- Layout Structure: Main content area with dynamically added, collapsible cards.
- UI Components:
 - Page Header: Title **Step 2: Section Architecture**.
 - Primary Action: A prominent, dashed-border button + **Add New Section**.
 - Section Card (Accordion Style): For each added section, generate a card containing:
 - **Section Title** (Text Input, e.g., "Reading Comprehension").
 - **Section Instructions** (Rich Text Editor).
 - **Section Duration** (Number Input - Minutes).
 - **Passing Threshold** (Number Input - Optional).
 - **Weight %** (Number Input).
 - **Action: Delete Section** icon (Trash bin).
 - Real-time Validation Footer (Sticky above Action Bar):
 - Displays: **Total Exam Weight: [X]%**.
 - **Logic:** This text must be Red if X is not exactly 100. It turns Green when exactly 100%.
 - Bottom Action Bar:
 - Secondary Button: **<- Back to Settings**
 - Primary Button: **Save Sections & Continue ->** (Disabled if total weight = 100%).

Page 3: /step-3-workspace (Content Workspace)

Purpose: Populate the sections with items from the item bank using a highly interactive Drag-and-Drop interface.

- Layout Structure: Split-Screen Interface (Left Sidebar vs. Main Workspace).
 - *RTL Note for Arabic:* Sidebar on the Right, Workspace on the Left.
- UI Components:
 - Sidebar: Item Bank Repository (30% width):
 - **Search Bar & Filter Icon** (Filter by CLO, Item Type, Difficulty).
 - **Item List:** Scrollable list of question cards. Each card displays the question snippet, type icon, and CLO tag.

- **Drag Handle:** Visible on hover to indicate the card can be moved.
- **Main Workspace (70% width):**
 - Displays the sections created in Step 2 as distinct visual "Drop Zones" (Containers).
 - **Dynamic Pooling Button:** Inside each section container, include an + **Auto-Pull from Blueprint** button. Clicking this opens a modal to select CLOs and the number of randomized questions to pull.
 - **Composite Item Rendering:** If a **COMPOSITE_CASE** item is dropped, it renders natively as a parent block with its child questions visibly nested underneath it.
- **Bottom Action Bar:**
 - **Secondary Button:** <- **Back to Sections**
 - **Primary Button:** **Save Content & Continue** ->

Page 4: /step-4-rubrics (Grading & AI Rubrics)

Purpose: Configure hierarchical rubrics for subjective questions (Essays, Audio, Video) and enable AI grading. *(Note: If no subjective questions exist, this page displays a "Not Applicable" state and allows skipping).*

- **Layout Structure:** Master-Detail View (Left list of items, Right rubric builder).
- **UI Components:**
 - **Left Panel (Master):** List of all subjective items added in Step 3. Selecting an item loads its rubric settings on the right.
 - **Right Panel (Detail / Rubric Builder):**
 - **Enable AI Auto-Grading** (Toggle Switch, prominent at the top).
 - **Rubric Table Structure:**
 - Columns for Performance Levels (Editable headers: e.g., Excellent, Good, Poor).
 - Rows for Dimensions (Parent Nodes).
 - Nested Rows for Sub-Dimensions (Leaf Nodes).
 - **Inline Editing:** Clicking any text cell transforms it into a text area to edit descriptors.
 - **Weight Inputs:** Numeric inputs next to each Dimension/Sub-dimension.
 - **Auto-Validation Indicator:** Similar to Step 2, a localized warning if $\Sigma \text{Weights}_{\text{sub}} = \text{Weight}_{\text{parent}}$.
 - **Bottom Action Bar:**
 - **Secondary Button:** <- **Back to Content**
 - **Primary Button:** **Save Rubrics & Continue** ->

Page 5: /step-5-dispatch (Targeting & Dispatch)

Purpose: Finalize exam scheduling and assign it utilizing the hybrid Groups/Tags targeting engine.

- Layout Structure: Two-column grid (Left: Targeting, Right: Scheduling).
- UI Components:
 - Left Column: Target Audience (Hybrid Resolver):
 - **Select Groups** (Multi-select Dropdown / Autocomplete): Lists institutional sections/cohorts.
 - **Targeting Logic Operator** (Radio Buttons):
 - **() ANY (OR)** - Students in the Group OR holding the Tag.
 - **() MUST MATCH (AND)** - Students in the Group AND holding the Tag.
 - **Select Profile Tags** (Tag Input Box / Autocomplete): Add tags like "Extra Time", "Advanced Track".
 - **Dynamic Preview**: A small text element that auto-updates: *"This exam will be assigned to approximately [X] students."*
 - Right Column: Scheduling & Availability:
 - **Start Date & Time** (Date/Time Picker).
 - **End Date & Time** (Date/Time Picker).
 - **Late Submission Policy** (Dropdown).
 - Bottom Action Bar:
 - Secondary Button: **<- Back to Rubrics**
 - Primary Button (Prominent, distinct color):  **Publish Exam**
(Submits final payload, updates database status to "Active/Scheduled", and redirects to the Exam Dashboard).

Path2: EvaliX Blueprint-First Flow

Architecture Model: Multi-Page Application (MPA) Target Audience: Academic coordinators, QA committees, and institutional instructors. Global UI Elements: Same as the default wizard (Top Nav, Progress Stepper, Toast Notifications).

Page 1: /blueprint-step-1-configuration (Exam Settings)

Purpose: Define exam identity and overall security parameters. *(This page is functionally identical to Step 1 in the standard wizard).*

- UI Components:
 - Exam Title, Course Code, General Instructions.
 - Total Exam Duration.
 - Sequential Navigation Locks (Completed Sections / Answered Questions).
 - Bottom Action Bar: Save & Continue ->

Page 2: /blueprint-step-2-matrix (The Blueprint Grid)

Purpose: The core of this flow. Instructors design the exam by mapping Course Learning Outcomes (CLOs) against cognitive levels or question counts.

- Layout Structure: A large, interactive data grid (spreadsheet-like UI) dominating the screen.
- UI Components:
 - Page Header: Title Step 2: Blueprint Specifications.
 - The Matrix Table:
 - Rows (Y-axis): List of officially mapped CLOs for the selected Course Code (e.g., CLO 1.1: Knowledge, CLO 2.1: Cognitive Skills). Include an + Add CLO button to fetch more from the syllabus.
 - Columns (X-axis): Item Types (e.g., MCQ, Essay) OR Cognitive Levels (e.g., Remember, Analyze).
 - Cells (Intersections): Number inputs. The instructor types the exact number of questions required for that specific CLO and Type.
 - Real-time Availability Tooltip (Critical Feature):
 - When hovering over or focusing on a cell, a tooltip must query the item bank and display: "Available in Bank: [X] questions".
 - If the instructor requests 5 questions but the bank only has 3, the cell border turns Red and a warning icon appears.
 - Summary Footer Panel (Sticky):

- Displays **Total Questions Requested: [X]**.
- Displays **Estimated Total Weight/Score: [Y]**.
- Bottom Action Bar:
 - Secondary Button: <- **Back to Settings**
 - Primary Button (Actionable): ⚙️ **Generate Exam from Blueprint**
-> (Disabled if there are red validation errors in the matrix).

Page 3: /blueprint-step-3-review (Auto-Generation & Audit)

Purpose: The system automatically executes Stratified Random Pooling based on the blueprint and presents the generated exam for human review.

- **Layout Structure:** Single column, list view grouped by CLOs or Sections.
- **UI Components:**
 - **Page Header:** Title **Step 3: Audit & Review Exam**.
 - **Generation Summary Banner (Top):** "Successfully generated [X] questions covering [Y] CLOs."
 - **Item List (The Generated Exam):**
 - Questions are displayed in readable cards.
 - Each card explicitly displays its metadata tags (e.g., **CLO 2.1, MCQ, Difficulty: Medium**).
 - **"Swap Item" Action (Crucial for Flexibility):**
 - Next to every generated question, place a  **Swap** button.
 - **Action:** Clicking it opens a modal showing alternative questions from the bank that match the *exact same CLO and parameters*. The instructor can click one to instantly replace the generated item, maintaining blueprint integrity.
 - **Bottom Action Bar:**
 - Secondary Button: <- **Edit Blueprint** (Destroys current generation and returns to grid).
 - Primary Button: **Approve Exam & Continue** ->

Page 4: /blueprint-step-4-rubrics (Grading & AI Rubrics)

Purpose: Configure rubrics for any subjective questions that were auto-pulled during the generation phase.

- **Behavioral Logic:**
 - If the blueprint only pulled objective questions (e.g., MCQs, True/False), this page renders a "Success State" message: *"No subjective questions detected. AI Grading and Rubrics are not required."*
 - If subjective items (Essays, Case Studies) were generated, this page loads the identical Master-Detail Rubric Builder defined in the standard wizard,

allowing the instructor to enable AI grading and adjust dimension weights.

- Bottom Action Bar:
 - Secondary Button: <- Back to Review
 - Primary Button: Save & Continue ->

Page 5: /blueprint-step-5-dispatch (Targeting & Dispatch)

Purpose: Finalize audience targeting and scheduling. *(This page is functionally identical to Step 5 in the standard wizard).*

- UI Components:
 - Hybrid Targeting Logic (Groups AND/OR Tags).
 - Scheduling Interface (Start/End times).
 - Bottom Action Bar:
 - Secondary Button: <- Back to Rubrics
 - Primary Button:  Publish Exam

"AI Path3: Express Mode" Flow

Architecture Model: Multi-Page Application (MPA) - Express Route (3 Pages). Optimal Use Case: In-class formative assessments, end-of-lecture pop quizzes, and quick memory refreshers. Global UI Elements: A simplified top progress stepper containing exactly 3 steps (Prompting, Review, Quick Dispatch).

Page 1: /express-step-1-prompt (The AI Prompt & Config)

Purpose: Feed the AI engine with the scientific material and required parameters to generate questions instantly.

- Layout Structure: Form input utilizing sequential, clean cards.
- UI Components:
 - Page Header: Title **Step 1: AI Generation Parameters**.
 - Content Source Card: A tabbed interface allowing the instructor to choose the input method:
 - **Tab 1 (Paste Text)**: A large Text Area to copy and paste a paragraph, article, or case study.
 - **Tab 2 (Select CLO/Topic)**: A dropdown to select a Course Learning Outcome (CLO) or a text input for a general topic (e.g., "Newton's Laws of Motion").
 - **Tab 3 (File Upload)**: A Drag & Drop zone to upload a PDF or slide deck for the model to parse.
 - Generation Parameters Card:
 - **Number of Questions**: A slider from 1 to a maximum of 20 questions (capped to ensure rapid API response times).
 - **Question Types**: Checkboxes to select combinations of: Multiple Choice, True/False, and Short Answer.
 - **Difficulty Level**: Radio buttons for Easy, Medium, Hard, or "Random Mix".
 - Bottom Action Bar: A prominent, stylized primary button (e.g., with a ✨ icon): ✨ **Generate Questions Now** ->. (Clicking this triggers a dynamic loading screen indicating that the AI is processing the request).

Page 2: /express-step-2-review (Review & Edit)

Purpose: Display AI outputs for the instructor to refine, edit, and approve correct answers in a single, frictionless screen.

- Layout Structure: Vertical, Inline-Editable List.

- UI Components:
 - Page Header: Title **Step 2: Review & Approval.**
 - Generated Question Cards:
 - Each generated question is displayed in its own distinct card.
 - *Inline Edit*: The instructor can click directly on the question text or any of the multiple-choice options to edit them instantly, mimicking a rich-text word processor.
 - *Correct Answer Selection*: The AI pre-selects the correct answer (visually highlighted in green). The instructor can override this by simply clicking the radio button of another option.
 - Item-Level Actions (Top right of each card):
 -  **Rephrase** Button: If the instructor dislikes a specific question, clicking this commands the AI to swap it with a newly generated alternative from the same context in seconds.
 -  **Delete** Button.
 - Institutional Save Option (Crucial for Item Bank growth):
 - A checkbox anchored at the bottom of the list: *"Save these generated questions to the Institutional Item Bank for future use."*
 - Bottom Action Bar:
 - Secondary Button: **<- Back to Prompt.**
 - Primary Button: **Approve Questions & Continue ->.**

Page 3: /express-step-3-dispatch (Quick Dispatch)

Purpose: Send the quiz immediately to students in the lecture hall or as a quick homework assignment with the absolute minimum number of clicks.

- Layout Structure: Extremely simplified, single-column screen (omitting the complex cross-tagging UI used in high-stakes exams to maximize speed).
- UI Components:
 - Page Header: Title **Step 3: Launch Express Quiz.**
 - Direct Targeting Card:
 - A quick dropdown to select the **Group / Section**. (*Smart Feature: The sections the instructor is currently scheduled to teach today should auto-populate at the top of this list*).
 - Quick Timers Card:
 - One-click preset buttons: **[Start Now (Open for 15 mins)]**, **[Start Now (Open until end of day)]**, or a standard **Set Custom Time** option.
 - Bottom Action Bar:
 - Giant Primary Button:  **Publish to Students Now.**
 - (*Post-Click Behavior: Upon publishing, the screen immediately displays a large, scannable QR Code and a short link. Students in the*

physical lecture hall can scan it with their phones to enter the quiz instantly).

Smart Item Bank Ingestion (AI Import)

Module: Item Bank Management (Not Exam Builder). Architecture Model: Multi-Page Application (MPA) - Ingestion Route (3 Pages). Purpose: Rapidly digitize legacy exams and unstructured documents into reusable, CLO-tagged database items. Global UI Elements: A module-specific top progress stepper (Upload, Audit & Tag, Success & Routing).

Page 1: /item-bank/import-upload (Upload & Parser Config)

Purpose: Receive the legacy document and provide the AI parser with contextual instructions.

- Layout Structure: Centered, modern upload interface.
- UI Components:
 - Page Header: Title **Step 1: AI Smart Ingestion**.
 - Drag & Drop Zone:
 - Large upload area supporting **.docx**, **.pdf**, and **.txt**.
 - Helper text: *"Drag and drop a legacy exam or question list here. Our AI will automatically extract and digitize the items."*
 - Parsing Settings (Card):
 - **Document Language**: Dropdown (Arabic, English, Bilingual).
 - **Pre-marked Answers**: Checkbox stating *"This document is an answer key (correct answers are highlighted, bolded, or underlined)."* (This feeds a specific prompt to the AI to auto-select correct options).
 - Bottom Action Bar:
 - Primary Button:  **Extract Questions ->**. (Triggers a dynamic loading state displaying processing steps: Reading text > Identifying item types > Mapping options).

Page 2: /item-bank/import-audit (Audit & Structuring Workspace)

Purpose: The critical human-in-the-loop interface. Allows the instructor to verify the AI's extraction against the original document and enrich the items with metadata before committing them to the database.

- Layout Structure: Split-Screen Interface (Left: Digital Editor, Right: Original Document Viewer).
 - *RTL Note for Arabic*: Digital Editor on the Right, Document Viewer on the Left.
- UI Components:
 - Page Header: Title **Step 2: Audit, Edit & Tag.**
 - Original Document Viewer (50% width):
 - Renders the uploaded PDF/Word file.
 - Text successfully identified as questions is lightly highlighted in yellow to serve as a visual anchor.
 - Digital Editor & Metadata (50% width):
 - Scrollable list of extracted items rendered as inline-editable cards.
 - Quick-Fix Tools per item:
 -  **Merge with Previous** (If the AI falsely split one question into two).
 -  **Split Item** (If the AI merged two separate questions).
 - Interactive radio buttons/checkboxes to manually correct the answer key if the AI missed it.
 - Bulk Tagging Toolbar (Crucial for Item Banking):
 - A sticky bar at the top of the list allowing multi-select.
 - Instructors can select 10 questions and apply a single **Course Learning Outcome (CLO)** or **Difficulty Level** to all of them simultaneously.
 - Bottom Action Bar:
 - Secondary Button: **<- Cancel Import.**
 - Primary Button: **Approve & Save to Item Bank ->.**

Page 3: /item-bank/import-success (Success & Intelligent Routing)

Purpose: Confirm successful database injection and offer the "Best of Both Worlds" bridge—allowing the user to either stay in the item bank or instantly launch an exam using the newly imported items.

- Layout Structure: Minimalist Success Screen.
- UI Components:
 - Success Graphic: A large green checkmark or animated success illustration.
 - Metrics Summary: *"Successfully added [X] new items to [Course Code] Item Bank."*
 - Action Routing (The Bridge):
 - Option A (Secondary Outline Button): **Done (Return to Item Bank)** - Ends the flow.

- Option B (Massive, Prominent Primary Button):  **Save & Create Exam Now.**
- *Developer Note for Option B:* Clicking this button redirects the user to the Exam Builder Wizard (Step 1). The backend must automatically create a new Exam Draft and pre-load all the items imported in this session into the draft's payload, skipping the manual drag-and-drop step.

Evalix

Search teachers, exams...

ENع

🔔

Dr. Essa
Institution Admin

MENU

Dashboard

Teachers

Exams

Item Review

Item Banks

Analytics

Curriculum

Settings

Curriculum Architecture

هيكل المنهج الدراسي

Manage the Course → Topic → Learning Objective hierarchy used for CLO mapping and accreditation reports

إدارة هيكل المقرر → الموضوع → الهدف التعليمي

+ Add Course

Courses

2

EHET611
Instructional Design

EHET620
Mobile Learning

+ Add Course

Topics

0

→ select a course

← Select a course first

Learning Objectives (CLOs)

0

→ select a topic

← Select a topic first

How CLO Mapping Works

When you create a question, select a Learning Objective from the Mapping tab. The question automatically inherits the CLO's Bloom's Taxonomy level and Learning Domain. Analytics reports (including NCAAAA accreditation exports) aggregate student performance by these inherited attributes — no manual taxonomy entry needed.

Evalix

Search exams, students...

ENع

🔔

Dr. Essa Almazroei
Teacher

MENU

Dashboard

Exams

Item Bank

Classes

Live Monitor

Students

Analytics

Settings

Classes

الفصول

Create classes and invite students to join them

التي الفصول وإلغ الطلاب للانضمام إليها

+ New Class

Show archived

فصل تدريبي

0 students

introduction 14658

0 students

Evalix

Search exams, students...

ENع

🔔

Dr. Essa Almazroei
Teacher

MENU

Dashboard

Exams

Item Bank

Classes

Live Monitor

Students

Analytics

Settings

Item Banks

بنوك الأسئلة

Institutional repositories and your own collaborative question banks

مستودعات المؤسسة وبنوك الأسئلة التعاونية

+ New Private Bank

Institution Banks (1)

My Private Banks (0)

Shared with Me (0)

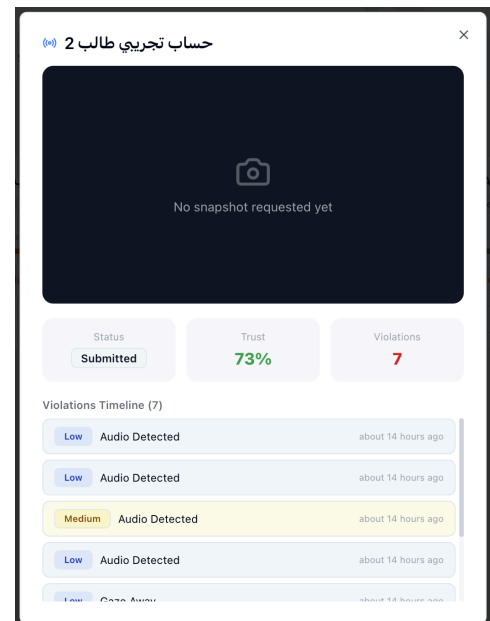
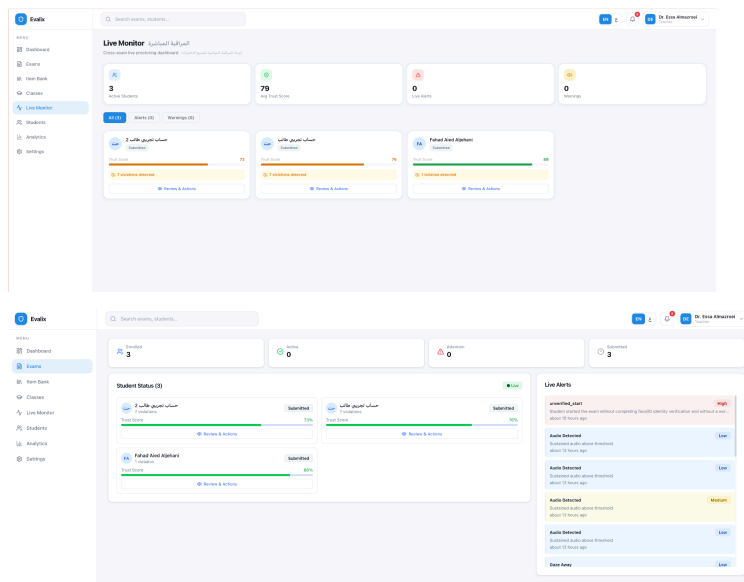
مقدمة في البرمجة

Editor

CS101

20 Items

Update 9 august 2026

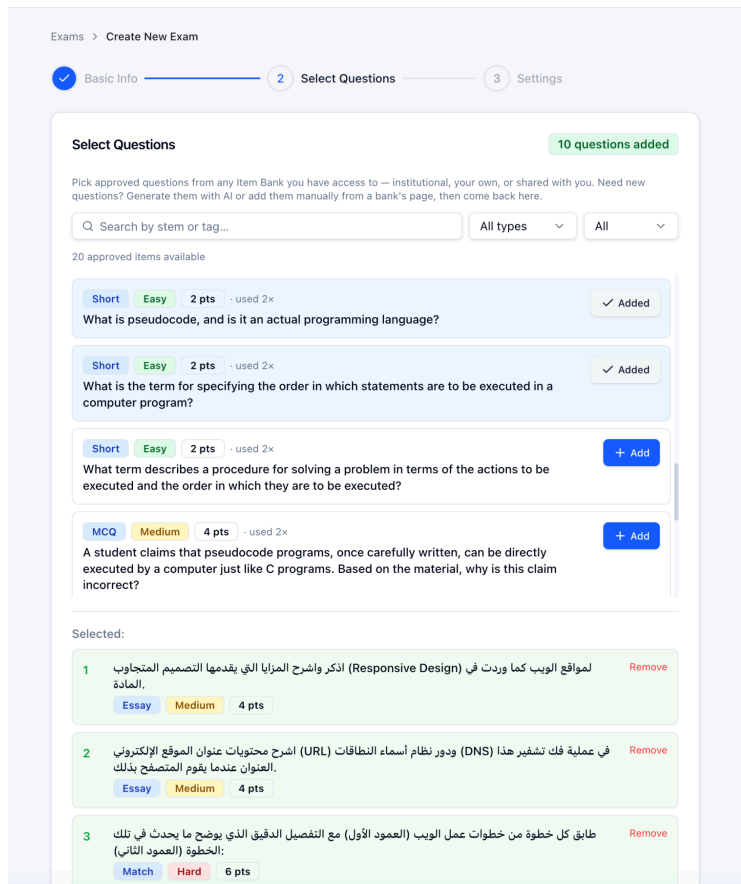


Ticket 1: Inconsistent Trust Score Calculation & UI Color Coding

- **Module / Screen:** Live Monitor Dashboard.
- **Issue Type:** Business Logic / UI Bug.
- **Steps to Reproduce:**
 - Navigate to the 'Live Monitor' for an active or submitted exam.
 - Compare the 'Trust Score' of multiple students who have the exact same number of violations.
 - Compare the color of the Trust Score progress bar in the main dashboard view versus the detailed student view.
- **Actual Result:**
 - **Logic:** Deductions are inconsistent. For example, two students each have 7 violations, but one has a Trust Score of 73% while the other has 76% or 78%.
 - **UI:** Color coding is conflicting. A 73% score shows a green progress bar in one view, but an orange bar in another view for a similar score.
- **Expected Result:**
 - **Logic:** Trust Score deductions must be standardized based on the severity weight of each violation (e.g., Low = -2%, Medium = -5%, High = -10%).
 - **UI:** The progress bar color thresholds must be globally unified across all components (e.g., < 50% = Red, 50% - 79% = Orange/Yellow, >= 80% = Green).
- **Priority:** High.

Ticket 2: Missing Media Evidence (Snapshots/Audio) for AI Violations

-  **Module / Screen:** Live Monitor -> Student Detail Modal.
-  **Issue Type:** Critical Bug.
-  **Steps to Reproduce:**
 - Open the 'Live Monitor' for an exam.
 - Click on a student who has recorded AI violations (e.g., "Audio Detected" or "Gaze Away").
 - Check the media player area at the top of the modal.
-  **Actual Result:**
 - The system successfully logs the violation type and timestamp in the timeline, but the media viewer displays a placeholder message: *"No snapshot requested yet."* No photographic or audio evidence is captured for the instructor to review.
-  **Expected Result:**
 - When the AI proctoring engine triggers a violation, it must automatically capture a snapshot (for visual violations) or an audio clip (for audio violations) and attach it to the timeline event payload.
 - Clicking on a specific violation in the timeline should instantly load the corresponding media evidence in the player.
-  **Priority:** Critical.



Ticket 3: Inability to Edit Question Points in Exam Builder (Step 2)

- 📌 Module / Screen: Exam Builder -> Step 2: Select Questions (Ref: 04.jpg).
- 🔴 Issue Type: UX/UI Enhancement / Business Logic Flaw.
- 📝 Description of the Issue: Currently, the point value of a question is strictly inherited from the Item Bank or the AI generation defaults and is rendered as static text. Instructors must have the flexibility to adjust the weight/points of a question specifically for the exam they are building, without being forced to accept the AI's default or altering the original asset in the Item Bank.
- ❌ Actual Result:
 - In the "Selected" items container (bottom of the screen), the points are displayed as static UI tags (e.g., 4 pts).
 - The user cannot click or modify these points. AI-generated questions enforce their default grades permanently.
- ✅ Expected Result (Required Changes):

1. **Inline Editable Points:** In the "Selected" list, convert the static points tag into a numeric input field (e.g., `<input type="number">`). The instructor should be able to click and type a new point value instantly.
 2. **Decoupled Values:** Editing the points in this view must only apply to the current exam payload. It must not overwrite the default point value stored in the master Item Bank.
 3. **Dynamic Total Counter (New UI Element):** Add a "Total Points" counter (e.g., **Total: 40 pts**) next to the green **10 questions added** badge at the top right. This counter must update in real-time as the user adjusts the individual points below.
- ⚡ **Priority: High.**

Exams > Create New Exam

1 Basic Info — 2 Select Questions — 3 Settings

Basic Information

Exam Title
اختبار الدوري الثاني

Class (optional — leave unset to assign to all your students)
فصل تجريبي

Only students enrolled in this class will be able to see and take this exam.

Subject
مقدمة في البرمجة

Total Marks
10

Passing Marks
5

Start Time
09/08/2026 02:15 PM

End Time
09/08/2026 02:30 PM

15 minutes — duration is calculated automatically from the start and end time.

Instructions (shown to students before they start)
e.g. Calculators are prohibited. Ensure your camera is active. Read each question fully before answering.

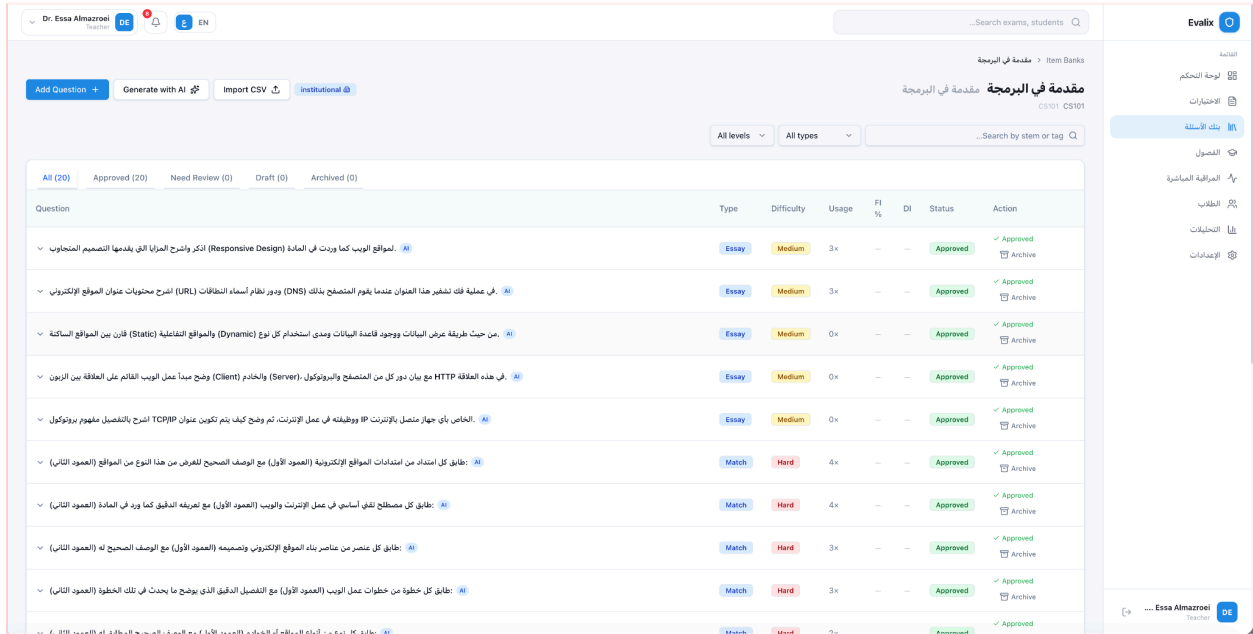
Next >

Ticket 4: Decouple "Availability Window" from "Exam Duration"

- 📌 Module / Screen: Exam Builder -> Step 1: Basic Info (Ref: 05 .png).
- 🔴 Issue Type: Critical Business Logic Flaw / UI Update.
- 📝 Description of the Issue: Currently, the system automatically calculates the exam's duration based on the difference between the "Start Time" and "End Time" (as explicitly stated in the blue alert box). This logic is fundamentally flawed for academic assessments. The time window during which an exam is *accessible* is distinct from the countdown timer a student gets *after* they begin.
- ❌ Actual Result:
 - "Availability Window" and "Exam Duration" are merged into a single concept.
 - If an instructor wants to leave an exam open for 24 hours so students can take it at their convenience, the system currently gives the student a 24-hour countdown timer, which invalidates the assessment.
- ✅ Expected Result (Required Changes):
 - Add a Dedicated Duration Field: Introduce a new numeric input field labeled **Exam Duration (Minutes)**. This field dictates the actual countdown timer the student will experience once they click "Start Exam".
 - Redefine Start/End Times: The **Start Time** and **End Time** fields must strictly function as the Availability Window (the timeframe when the "Start

Exam" button is visible/active for the student).

3. Remove Auto-Calculation: Remove the blue info box ("*15 minutes — duration is calculated automatically...*") as this logic is no longer valid.
 4. Enforce Cut-off Logic (Backend Check): The backend must handle late starters. If a student starts the exam 10 minutes before the **End Time**, but the **Exam Duration** is 60 minutes, the exam should forcefully auto-submit at the exact **End Time**, regardless of the remaining duration timer.
- ⚡ Priority: Critical.



Ticket 5: Broken RTL Layout & Column Ordering on "Item Bank" Page

- 📌 **Module / Screen:** Item Bank.
- 🔴 **Issue Type:** UI/UX Bug (Localization / RTL Support).
- 📝 **Description of the Issue:** When the Arabic interface is active, the "Item Bank" page fails to properly mirror its layout structurally (Right-to-Left). While the Arabic text itself is right-aligned within its containers, the flexbox/grid containers and the table column order are still behaving as LTR (Left-to-Right). This creates an inconsistent and confusing experience compared to the "Exams" page, which handles RTL perfectly.
- ❌ **Actual Result:**
 - Table Columns: Ordered Left-to-Right. The "Question" column is on the far left, and the "Action" column is on the far right.
 - Tabs & Buttons: The tabs (All, Approved) and the action buttons (Add Question, Generate with AI) are anchored to the left side of the screen.
- ✅ **Expected Result (Matching Exams page):**
 - Global RTL Property: The main content wrapper for the Item Bank must properly inherit the `dir="rtl"` HTML attribute.
 - Table Columns: Must flow Right-to-Left. The "Question" column should be on the far right (closest to the sidebar), cascading leftwards, ending with the "Action" column on the far left.
 - CSS Logical Properties: Ensure developers are using CSS logical properties (e.g., `margin-inline-start`, `flex-direction: row` reacting to

`dir="rtl"`) instead of hardcoded physical properties (like `margin-left` or `float: left`) for the tabs and action buttons.

- ⚡ Priority: High (Visual consistency and localization standard).

Create Item < Bank < Item Banks

Basic CLO Mapping Settings

Question Type

Multiple Choice (MCQ) Open Choices	Multiple Response (MRQ) Open Choices	True / False Limited Choices	Short Answer Complete
Essay Series	Fill in the Blank Complete	Matching Matching & Ordering	Ordering Matching & Ordering
Coding Challenge Advanced	File Submission Advanced		

Question Stem

Σ Math ⚗ Chemistry AI Assist

Enter your question here...

Answer Options

☐ A	Option A	Σ ⚗	🗑
☐ B	Option B	Σ ⚗	🗑
☐ C	Option C	Σ ⚗	🗑
☐ D	Option D	Σ ⚗	🗑

+ Add Option

Click the circle to mark the correct answer(s)

Ticket 6: Replace Plain Text Inputs with Rich Text Editor (RTE) Incorporating Equation Tools

- 📌 **Module / Screen:** Item Bank -> Create Item.
- 🔴 **Issue Type:** UX/UI Enhancement / Core Feature Update.
- 📝 **Description of the Issue:**
The current method for inputting Math and Chemistry formulas relies on standalone, external buttons placed outside the text input fields. This approach is impractical, disrupts the user workflow, and restricts instructors from using basic text formatting (bold, italics, lists, inline images) alongside their equations.
- ❌ **Actual Result:**
 - The "Question Stem" and "Answer Options" use basic plain text inputs (`<textarea>` and `<input>`).
 - The Math (Σ) and Chemistry (⚗) triggers are floating buttons disconnected from the text flow, making inline equation editing clunky.

-  Expected Result (Required Changes):
 1. Implement a WYSIWYG/Rich Text Editor (RTE): Replace the plain text fields in both the "Question Stem" and all "Answer Options" with a lightweight Rich Text Editor (e.g., Quill.js, TinyMCE, or CKEditor).
 2. Integrated Toolbar: Include a standard formatting toolbar (Bold, Italic, Underline, Lists, Image Upload) at the top of the Question Stem, and a compact/inline toolbar for the Answer Options.
 3. Embed Equation Tools inside the Toolbar: Move the "Math" and "Chemistry" tools *inside* the RTE toolbar as dedicated formatting icons. Clicking them should open the equation builder modal, and upon saving, insert the formula directly inline with the rest of the formatted text.
-  Priority: High (Crucial for STEM subject adoption and overall usability).

marks 4
Medium
Mcq

Q11. A student claims that pseudocode programs, once carefully written, can be directly executed by a computer just like C programs. Based on the material, why is this claim incorrect

Pseudocode programs are not executed on computers; they merely help you think out a program before writing it in a language like C
☐

Pseudocode can only represent selection structures, not iteration structures
☐

Pseudocode requires a special compiler that translates it into C automatically
☐

Pseudocode statements must always be converted into flowcharts before they can run
☐

- 📌 Module / Screen: Student Assessment Interface (Exam Taking View).
- 🔴 Issue Type: UI Bug / CSS / Localization.
- 📝 Description of the Issue: Currently, user-generated exam content (Question stems and Answer options) strictly inherits the global `dir` attribute of the application's UI. This breaks readability when a student takes an English exam while using the Arabic UI (text aligns right, and punctuation like question marks are misplaced at the beginning of the line), or vice versa. Exam content must retain its natural directionality regardless of the outer UI language.
- ❌ Actual Result:
 - English text in an Arabic UI is forced into RTL alignment.
 - Punctuation marks at the end of English sentences render at the far left (start) of the line.
- ✅ Expected Result (Required Changes):
 - Implement `dir="auto"`: Apply the HTML attribute `dir="auto"` directly to the text wrapper elements of the Question Stem and the Answer Options. This commands the browser to auto-detect the direction based on the first strongly-typed character in the string.
 - CSS Logical Properties: Ensure the CSS for these text blocks uses `text-align: start`; rather than hardcoded `left` or `right`. This ensures the text will correctly align to the left for English and right for Arabic, based on the computed direction from `dir="auto"`.
- ⚡ Priority: High (Directly impacts readability and student performance).



marks 6
Hard
Matching

Q1. طابق كل امتداد من امتدادات المواقع الإلكترونية (العمود الأول) مع الوصف الصحيح للغرض من هذا النوع من المواقع (العمود الثاني):

Match	Term
مواقع الغرض منها تقديم معلومات عن الجيش	مواقع تنتهي بالرمز .com
مواقع حكومية لتقديم المعلومات والخدمات التي تصدرها الوكالات الحكومية والإدارات	مواقع تنتهي بالرمز...
مواقع الغرض منها الدفاع أو الترويج لرأي الفرد أو وجهة نظر المجموعة	مواقع تنتهي بالرمز .gov
مواقع تجارية صممت لغرض بيع المنتجات أو الخدمات مثل www.noon.com	مواقع تنتهي بالرمز .mil
مواقع حكومية لتقديم المعلومات والخدمات التي تصدرها الوكالات الحكومية والإدارات	مواقع تنتهي بالرمز...

Ticket 8: UI Overlap / Text Overflow in Matching Question Component

- 📌 Module / Screen: Student Assessment Interface -> Matching Question (Ref: [Screenshot 1448-02-26 at 2.39.57 PM.png](#)).
- 🔴 Issue Type: UI Bug / CSS Responsive Issue.
- 📝 Description of the Issue: In the "Matching" question type, when the text inside the "Match" dropdown options is excessively long, the dropdown container expands horizontally beyond its allocated grid/flex space. This causes it to visually overlap and cover the static text in the "Term" column, breaking the layout and making the content unreadable.
- ❌ Actual Result:
 - The left column (Match dropdowns) ignores layout boundaries and overlaps the right column (Terms).
 - Long text is not truncated or wrapped properly.
- ✅ Expected Result (Required Changes):
 - Layout Constraints: Enforce strict column widths using CSS Grid or Flexbox (e.g., applying `minmax(0, 1fr)` to ensure the container doesn't grow beyond its parent's width). Add a fixed `gap` between the columns.
 - Text Truncation (Closed State): The selected text inside the dropdown

should be truncated with an ellipsis if it exceeds the maximum width (`overflow: hidden; white-space: nowrap; text-overflow: ellipsis;`).

3. Custom Dropdown Wrap (Open State): If using a native HTML `<select>`, long options will force the width. Consider using a custom UI dropdown component (like Radix UI or headless UI) that allows long text within the dropdown menu to wrap into multiple lines (`white-space: normal;`) instead of stretching horizontally.
- ⚡ Priority: High (Directly impacts the student's ability to read and answer the question).

Multiple Choice (MCQ)
Open Choices

Multiple Response (MRQ)
Open Choices

True / False
Limited Choices

Short Answer
Complete

Essay
Series

Fill in the Blank
Complete

Matching
Matching & Ordering

Ordering
Matching & Ordering

Coding Challenge
Advanced

File Submission
Advanced

Question Stem

Math

Chemistry

AI Assist

Enter your question here...

Explanation

Math

Chemistry

Why is this the correct answer? Shown with the answer key.

Optional. Supports math and chemistry.

Scoring Rubric

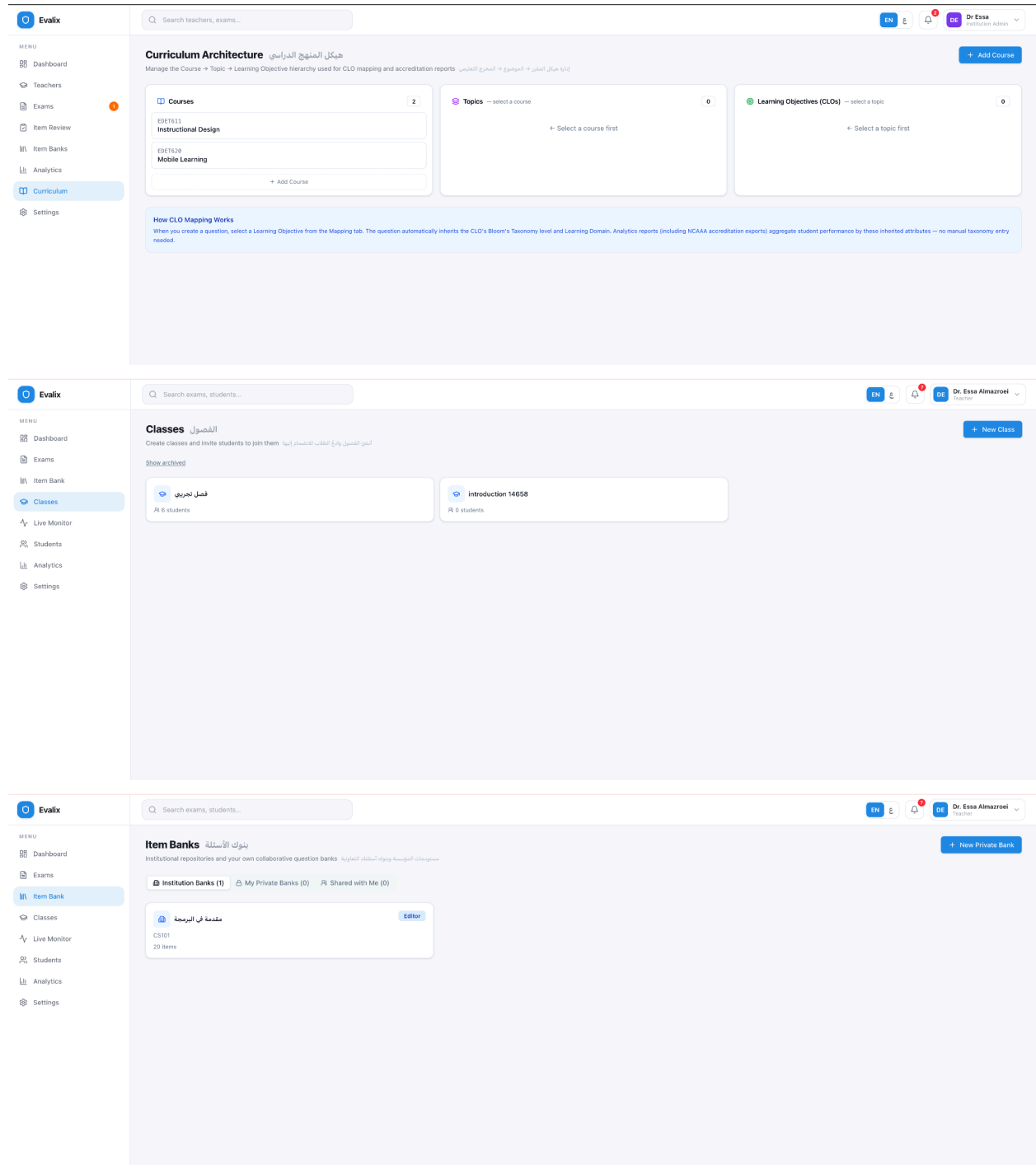
Dimension	Excellent (4)	Good (3)	Fair (2)	Poor (1)	Weight %
Content Accuracy	Describe...	Describe...	Describe...	Describe...	25
Critical Thinking	Describe...	Describe...	Describe...	Describe...	25
Structure & Clarity	Describe...	Describe...	Describe...	Describe...	25
Evidence & Examples	Describe...	Describe...	Describe...	Describe...	25

Ticket 9: Implement Flexible Hierarchical Scoring Rubric for Essay Questions

- 📌 Module / Screen: Item Bank -> Create Item (Essay Type) - Ref: [image_f7287e.jpg](#).
- 🔴 Issue Type: Core Feature Missing / UX/UI Redesign.
- 📝 Description of the Issue: The current "Scoring Rubric" component is implemented as a rigid, static table with hardcoded dimensions (e.g., Content Accuracy, Critical Thinking). As per the EvaliX system architecture, rubrics for subjective questions must be fully dynamic and hierarchical to support complex grading matrices and precise AI evaluation. Instructors must be able to define custom dimensions and nest sub-dimensions within them.
- ❌ Actual Result:
 - 1. The rubric rows (Dimensions) and columns (Performance Levels) are hardcoded and cannot be added, renamed, or deleted by the user.
 - 2. There is no UI support for a hierarchical structure (Parent Nodes / Leaf Nodes).
- ✅ Expected Result (Required Changes):
 - 1. Dynamic Grid: Allow users to freely add, delete, and rename both the Columns (Performance Levels, e.g., Excellent, Good) and the Rows (Dimensions).
 - 2. Hierarchical Structure (Sub-dimensions): Add a UI mechanism (such as an

"Add Sub-dimension" button under each main row) that allows nesting child criteria under a parent dimension.

3. **Weight Validation Logic:**
 - The total **Weight %** of all parent dimensions must automatically validate to 100%.
 - If sub-dimensions are added, the sum of the sub-dimension weights must strictly equal the weight of their parent dimension. Display real-time validation warnings (Red/Green text) if the math does not align.
 4. **Enable AI Auto-Grading Toggle:** Include a prominent toggle switch above the rubric table (**Enable AI Auto-Grading**) to link this specific rubric structure to the AI grading engine.
- ⚡ **Priority: High** (Essential for subjective grading accuracy and institutional compliance).



Ticket 10: Standardize 'Curriculum' and 'Classes' Architecture to Match 'Item Banks' (Institution, Private, Shared)

- 📌 Module / Screen: Curriculum ([image_6d11df.jpg](#)) & Classes ([image_6d0e9a.jpg](#)) vs. Item Banks ([image_6d0de1.jpg](#)).
- 🟡 Issue Type: Core Architectural Update / UI Refactor.
- 📝 Description of the Issue: Currently, the 'Curriculum' and 'Classes' modules utilize a flat data structure, lacking the robust collaborative and scoping features present in the 'Item Banks' module. To fully support scalable institutional workflows, both 'Curriculum' and 'Classes' must adopt the exact same tenancy, ownership, and sharing logic used in the 'Item Banks'.
- ❌ Actual Result:
 1. 'Curriculum' and 'Classes' screens only show a flat list of created entities without clear ownership separation (Institution vs. Private).
 2. There is no mechanism to share a specific Class or Curriculum structure with other teachers.
- ✅ Expected Result (Required Changes):
 1. UI Tabbed Navigation: Implement the identical three-tab layout from the Item Banks across both the 'Curriculum' and 'Classes' main screens:
 - **Institution Curriculums / Classes** (Global read-only access for all teachers, managed by admins/coordinators).
 - **My Private Curriculums / Classes** (Owned solely by the logged-in teacher).
 - **Shared with Me** (Items owned by other teachers but shared with the logged-in user).
 2. Backend / DB Schema Updates: Update the entities for **Curriculum** (and its nested CLOs/Topics) and **Class** to include **owner_id**, **institution_id**, and support a many-to-many sharing relationship table.
 3. Sharing Mechanism: Add a "Share" action to private curriculums and classes, triggering a modal that allows the owner to invite other registered teachers (via email or internal user search) with specific permissions (e.g., View only, or Edit).
- ⚡ Priority: High (Impacts database schema and core collaborative features).